# Password Cracking with Practical Quantum Computing

Rafael Cenzano Harry Hargreaves Jason Kim Timofey Tylik Joshua Youngbar

***Abstract*—Ever since the introduction of quantum computation there has been discussion of the possible consequences to cybersecurity. A key discussion point has been Shor's algorithm, allowing for the factoring of any natural number, with experimental quantum implementations being attempted as early as 2001. Shor's algorithm, if implemented fully, can be the key to breaking modern day encryption keys due to their heavy reliance on prime factorizations. However, this is unlikely to happen soon due to the many hardware restrictions on modern day quantum computers, so in this paper we focus on a different approach - Grover's algorithm. This allows for attacks given a list of commonly used passwords.**

***Index Terms*—Grover's algorithm, cybersecurity, quantum computing**

## I. Introduction

Quantum computing introduces new vulnerabilities to classical authentication methods by enabling more efficient attacks on cryptographic hashes. Grover's algorithm [1] offers a quadratic speedup for unstructured search problems and, therefore, can reduce the time complexity of brute-force password cracking from $O(N)$ to $O(\sqrt{N})$. This makes hashed passwords, which are a widely used method of securing credentials, vulnerable in a post-quantum world.

Prior work has explored this risk in theory. For example, Dürmuth et al. [2] modeled quantum password-guessing attacks using Grover's algorithm under idealized, error-free conditions. These studies confirmed the potential quantum advantage but did not account for the limitations of today's hardware, which is constrained by qubit count, noise, and lack of full error correction.

In our paper, we address that gap by evaluating how closely practical quantum systems can approach the theoretical performance of Grover's algorithm in a password-cracking context. We focus on a fixed-user attack, simulating the inversion of a hashed password drawn from a realistic candidate list. All experiments were conducted using Qiskit and AerSimulator, with quantum circuits implemented at the gate level.

Our approach integrates realistic hardware constraints, including noise models based on IBM's quantum backends, basic quantum error correction (QEC), and zero-noise extrapolation (ZNE). This allows us to assess the performance degradation introduced by noise and quantify the gap between theoretical and practical quantum attacks on password security.

## II. Background

### A. Grover's Algorithm

Grover's Algorithm, introduced by Lov Grover in 1996 [1], is a quantum search algorithm that provides a quadratic speed-up over classical unstructured search. In a classical setting, finding a target item among $N$ possibilities requires $O(N)$ queries. Grover's Algorithm reduces this to $O(\sqrt{N})$ queries by leveraging quantum superposition and interference to amplify the probability amplitude of the correct solution.

The algorithm begins by preparing a uniform superposition over all possible states. It then repeatedly applies two operations: an oracle, which flips the phase of the target state, and a diffuser (or inversion about the mean), which amplifies the target state's amplitude relative to the others. Repeating these operations approximately

$$k = \left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor$$

times maximizes the probability of measuring the correct result.

Grover's Algorithm is particularly relevant to password cracking, where an attacker must search an unstructured space of possible passwords. However, practical use is limited by the number of available qubits and the presence of noise and gate errors in quantum hardware, necessitating the use of error correction or mitigation methods.

In this project, Grover's algorithm is applied to a simulated password search scenario to evaluate how closely real-world performance approaches theoretical expectations.

### B. Towards Quantum Large-Scale Password Guessing on Real-World Distributions

Reference [2] provided the foundation for our project work. The paper discussed using Grover's Algorithm to provide a square root speed-up in the searching process when matching an attacker's hashed result and an unsorted hashed password dataset. The paper provided a purely theoretical mathematical model for the speed-up that could be achieved using a quantum computer with no errors that would interfere with the ability to use Grover's Algorithm. The paper acknowledged the limitation on the number of qubits available. In the modern day, the Rensselaer IBM Quantum System provides a little over double the qubits as the paper had [2]. However, it is still below what would be required to have a fully practical application of Grover's algorithm for this use case.

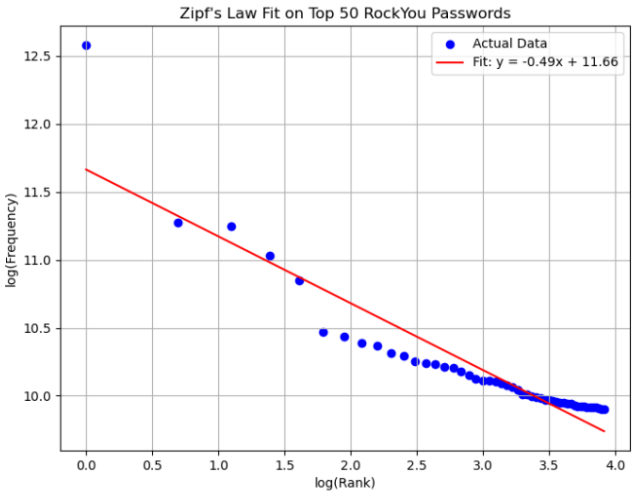


Fig. 1. Fit of RockYou Data with Zipf's Law

The paper focuses on two use cases, a fixed user attack and large scale attack. The fixed user attack is for the case where there is a singular password required, for example, a password to unlock a cryptographic key to decrypt data on a hard drive or an admin password for a system. The large-scale attack is the scenario that often impacts users of SaaS products, where the provider is hacked or unintentionally leaks their databases containing user logins. In this scenario, an attacker has a database of hashed passwords and must match passwords to gain login credentials to a system.

### C. ZIPF's Law

In this work we utilize a dataset closely modeled from the 2009 Rockyou password leak. This dataset is intended to closely follow Zipf's Law as this is a key property of many password leaks. Zipf's Law, as described in [3], claims that the frequency of a word in a natural language is inversely proportional to it's frequency rank and closely follows the relation $f(r) \propto \frac{c}{r^a}$. This is shown to be widely applicable to password datasets in [4] and that due to constraints of natural language and the human mind, most password datasets will have frequencies that closely follow the Zipf's Law relation. This unique property of password data sets allows for the number of guesses from the $N$ most common passwords to be reduced from $\frac{N}{2}$ to $\frac{1-s}{2-s} \cdot N$, where $s$ is the slope of the Zipf's Law line on a log-scale as shown in [2]. As mentioned above, our data closely fits Zipf's law as shown in Fig. 1. The $R^2$ value of the graph is 0.857, indicating a relatively good fit.

## III. Motivation

The motivation behind this paper is two-fold. First, we would like to provide a real quantum implementation of and verify the theoretical results of the algorithm in [2]. This was done via comparing a Grover's algorithm implementation to a classical benchmark and is discussed in the following section. Second, we sought to incorporate error and error prevention and correction models into our code. This allows us to discuss the limitations of modern quantum computing beyond the number of available qubits that may be holding back this attack strategy.

## IV. Methods

### A. Classical

To provide a benchmark for the performance of the quantum algorithm discussed in the following section, we use a classical algorithm. This algorithm simulates a *fixed-user attack* by iterating through a provided list of the top $N$ passwords, ordered by frequency. It then hashes the respective password and compares it to the hash of the password we are attacking. Throughout this work, all hashes will be SHA-1 unsalted hashes as this is a common practice in academic literature. When a matching hash is found, the plain text password is returned. If a match is not found, that password is disregarded as over a large sample size the top $N$ passwords cover a sufficiently large percentage of a data set.

To then construct a sufficiently accurate benchmark of the number of attempts required to crack a password, we run the *fixed-user attack* algorithm on 10,000 randomly selected users. Then the number of guesses is averaged. Aside from providing a key benchmark for comparing the quantum algorithm to the existing classical attack strategies, this allows us to confirm the relation provided in [2]. According to [2], we can expect to have an average number of guess equal to $\frac{1-s}{2-s} \cdot N$ given a set of the top $N$ passwords, where $s$ is the slope of the Zipf's Law line. Given the model of our data we expect 16.89 guesses per password on our average. When the benchmarking code was run, the average number of guesses was 17.02. While not perfect, this is a reasonably close benchmarking value when compared to the theoretical.

### B. Quantum

This project simulated a *fixed-user password attack* using Grover's algorithm. The objective was to evaluate the number of Grover iterations required to correctly identify a password from a finite list with high probability. All experiments were implemented in Python using the Qiskit library, executed in a Google Colab environment. Simulations were run using Qiskit's `AerSimulator` backend. The experiment was modeled on a noise-free simulator, and a simulator matching the error and noise rates of the IBM Rensselaer system.

A list of 50 most likely passwords was constructed, with a password selected from the list and hashed using the SHA-1 algorithm matching the algorithm used in the data from the LinkedIn 2012 hack. One password was selected as the target, and its corresponding hash served as the reference value.

Each password in the list was assigned a unique index from 0 to $N-1$, which was then converted into a binary string of length 6 to match the number of data qubits. This bitstring served as the quantum representation of the password in the superposition. After measurement, the output bitstring was reversed to account for Qiskit's little-endian qubit ordering, and then mapped back to its corresponding password by

index. This mapping ensured consistent interpretation between classical and quantum representations of the password space.

The Grover circuit used a total of 7 qubits: 6 data qubits to encode a search space of size $2^6 = 64$, and 1 ancilla qubit for phase kickback. The oracle was built using multi-controlled X (MCX) gates, conditioned to detect the target bitstring. To apply the phase flip required by Grover's algorithm, the ancilla was prepared in the $|-\rangle$ state by applying an X gate followed by a Hadamard gate. After the MCX operation, the ancilla was uncomputed to return it to $|0\rangle$. The diffuser (inversion about the mean) was constructed using Hadamard and X gates on all data qubits, followed by another MCX and Hadamard layer.

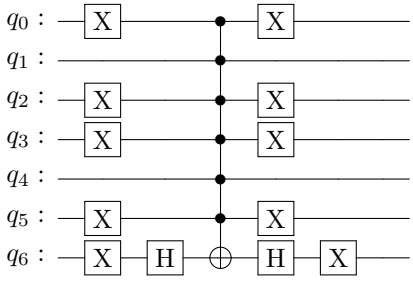


Fig. 2. Oracle subcircuit for marking the target bitstring (010010) using a multi-controlled X gate and ancilla qubit prepared in the $|-\rangle$ state.

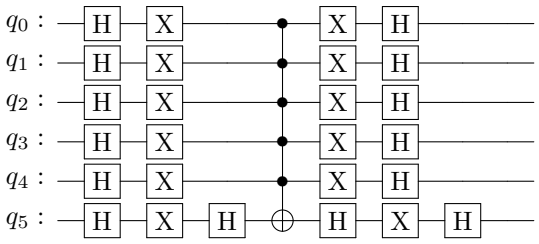


Fig. 3. Diffuser subcircuit (inversion about the mean) applied after the oracle in each Grover iteration. The circuit uses Hadamard and X gates to reflect about the average amplitude of all states, followed by a multi-controlled Z gate (implemented here as a multi-controlled X with surrounding Hadamards).

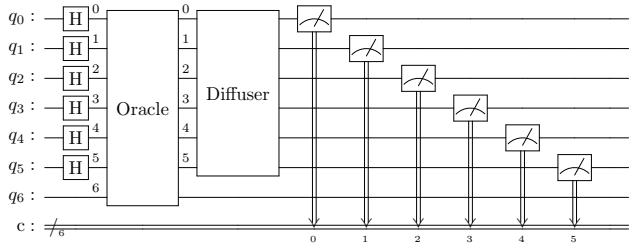


Fig. 4. Full Grover circuit with one iteration, composed of a uniform superposition initialization, a phase oracle subcircuit, and a diffuser. The circuit operates on six data qubits and one ancilla qubit, which is used for the controlled phase flip in the oracle.

The number of Grover iterations was chosen based on the standard formula:

$$k = \left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor$$

where $N = 50$ is the number of candidate passwords, yielding $k = 6$ iterations.

Four experimental configurations were tested to analyze the effect of iteration count and shot count:

1) 100 shots with 1 Grover iteration
2) 100 shots with 6 Grover iterations
3) 1 shot with 1 Grover iteration
4) 1 shot with 6 Grover iterations

In each trial, the quantum register was measured, and the result with the highest occurrence (in multi-shot experiments) was interpreted as the algorithm's predicted solution. The resulting password candidate was hashed and compared to the target hash to verify correctness. Additionally, the simulation time and confidence—defined as the proportion of correct results over the total number of shots—were recorded to evaluate Grover's effectiveness in each configuration.

Noise was introduced to the simulation through `QiskitRuntimeService`, after providing an API token:

```
service = QiskitRuntimeService()
```

Noise models can then be extracted from real-world quantum computers, based on the latest calibration data.

```
backend = service.backend("ibm_rensselaer")
noise_model = NoiseModel.from_backend(backend)
```

`AerSimulator` uses `noise_model`, and the noisy quantum circuit is compiled and ran:

```
sim = AerSimulator(noise_model=noise_model)
passmanager = generate_preset_pass_manager(
    optimization_level = 3,
    backend = sim
)
compiled = passmanager.run(qc)
result = sim.run(compiled, shots=shots)
         .result()
```

### C. Zero-Noise Extrapolation (ZNE)

We employ zero-noise extrapolation (ZNE) to approximate Grover's algorithm's ideal (error-free) performance on noisy quantum hardware. ZNE is an error mitigation technique in which a quantum circuit is executed under scaled noise levels, and the observed results are extrapolated back to an effective zero-noise setting. We implemented ZNE using Qiskit's `AerSimulator`, applying depolarizing noise scaled by factors of 1.0, 2.0, and 3.0 to simulate increasing gate error rates. The base error rates used were 1% for single-qubit gates and 3% for two-qubit gates.

For each noise level, we executed the Grover circuit with a fixed number of shots and recorded the most frequent measurement outcome. The proportion of this outcome across all shots was defined as the success confidence. These confidence

values were plotted against their corresponding noise scale factors, and a linear regression was performed. The y-intercept of the fitted line at zero noise provided the extrapolated confidence, which estimates success if the circuit were executed on a noiseless quantum processor. This helped us to assess the algorithm's efficacy, independent of hardware limitations.

To enhance the robustness of Grover's algorithm under noise, we implemented a basic quantum error correction scheme using the three-qubit bit-flip code. This QEC strategy encodes a single logical qubit into three physical qubits, allowing the system to correct for a single bit-flip error.

Two helper functions were developed for this purpose: `encode_bit_flip` and `decode_bit_flip`. The encoding function applies two CNOT gates from the original qubit to two auxiliary qubits, creating a three-qubit entangled state. This replicates the logical state across multiple physical qubits. After quantum operations are performed, the decoding function re-applies the same CNOT gates, followed by a majority vote using a Toffoli (CCX) gate to recover the original logical qubit from the possibly corrupted triplet.

This bit-flip code provides limited but meaningful error resilience, especially in the presence of dominant X (bit-flip) errors. While not a full fault-tolerant scheme, this implementation demonstrates how small-scale error correction can be integrated into password-cracking quantum circuits and serves as a foundation for scaling to more sophisticated codes in future work.

## V. Evaluation

The performance of Grover's algorithm was evaluated under four test conditions: 1 shot with 1 iteration, 100 shots with 1 iteration, 1 shot with 6 iterations, and 100 shots with 6 iterations. Each experiment simulated a fixed-user attack scenario, targeting a specific password from a list of 50 hashed passwords. The password list was constructed using the 50 most probable passwords following a ZIPF distribution, based on real-world password frequency data. The target hash was generated using the SHA-1 algorithm to reflect the format of the 2012 LinkedIn password leak.

Each Grover circuit consisted of 7 qubits: 6 data qubits to encode the index of the candidate password and 1 ancilla qubit used to implement the oracle's phase flip via quantum phase kickback. The oracle was constructed to mark a single target bitstring, and a diffuser circuit was used to amplify its amplitude. The number of Grover iterations in the 6-iteration configurations was chosen based on the theoretical optimum,

$$k = \left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor,$$

where $N = 50$, yielding $k = 6$.

In the configurations using only 1 Grover iteration, both 1-shot and 100-shot tests produced consistent results, with the correct password recovered in approximately 16% to 22% of 100-shot trials. This limited success is expected, as a single Grover iteration provides only partial amplitude amplification of the target state.

In contrast, tests using 6 Grover iterations were significantly more successful. The 1-shot, 6-iteration configuration produced correct results in 100% of runs, and the 100-shot configuration achieved success rates between 98% and 100%. These results closely match theoretical predictions, which suggest that Grover's algorithm achieves its maximum success probability after approximately $\left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor$ iterations in an ideal, noise-free setting [2].

These findings validate the expected quadratic speed-up of Grover's algorithm in the context of unstructured search. While low iteration counts produce suboptimal amplification, the correct number of iterations yields nearly guaranteed recovery of the target password.

### A. Zero-Noise Extrapolation Results

We applied zero-noise extrapolation to three test cases to estimate how Grover's algorithm would perform in an ideal, noise-free setting. These tests targeted passwords commonly found in real-world leaks: `qwerty`, `basketball`, and `password`. For each target, confidence levels were recorded at noise scale factors of 1.0, 2.0, and 3.0. A linear regression was applied, and the extrapolated success probability at zero noise was calculated.

The extrapolated confidence was 94.33% for `qwerty`, 92.67% for `basketball`, and 84.67% for `password`. These results demonstrate that Grover's algorithm retains high success probability in error-free simulations, consistent with theoretical expectations. Figures 5, 6, and 7 show the corresponding plots.

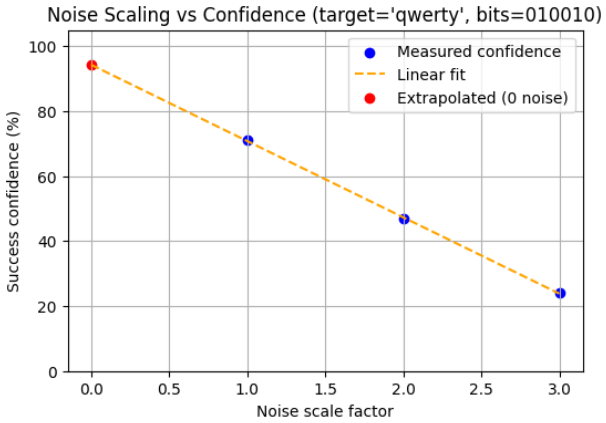


Fig. 5. ZNE result for `qwerty` (bitstring = 010010). Extrapolated zero-noise confidence = 94.33%.

## VI. Conclusion/Discussion

To better understand the gap between real and ideal execution, we applied zero-noise extrapolation (ZNE) to several target passwords. The results confirmed that Grover's algorithm approaches its theoretical behavior even in noisy environments when error mitigation is used. Extrapolated confidence levels above 84% across all test cases highlight the viability of

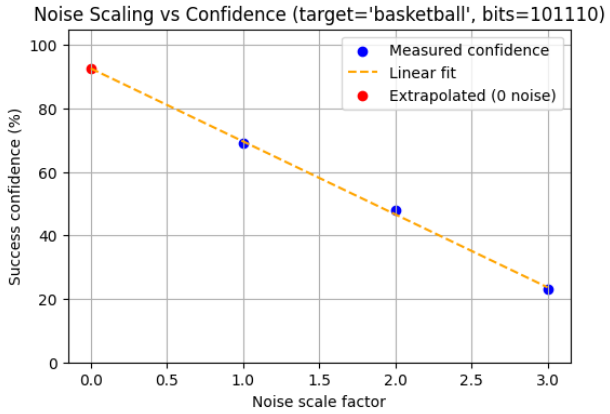


Fig. 6. ZNE result for `basketball` (bitstring = 101110). Extrapolated zero-noise confidence = 92.67%.

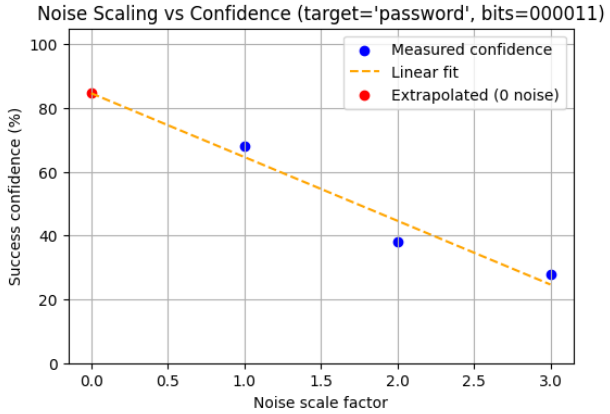


Fig. 7. ZNE result for `password` (bitstring = 000011). Extrapolated zero-noise confidence = 84.67%.

using quantum amplitude amplification with ZNE on near-term devices. These findings demonstrate that meaningful quantum advantage in password cracking can be assessed without fault-tolerant hardware by simulating ideal conditions through classical extrapolation.

To extend this research, we propose several directions:

- Expanded QEC Techniques: Moving beyond the bit-flip code to incorporate more general QEC schemes, such as phase-flip codes or surface codes, could better protect against a broader range of quantum errors.
- Real Hardware Experiments: Executing these circuits on physical quantum computers will validate our simulation results and help measure performance under live noise conditions.
- Scaling to Larger Password Spaces: Our experiments were limited to 50 candidate passwords due to qubit constraints. Expanding to larger search spaces will test the scalability of Grover's algorithm and its susceptibility to increased noise and circuit depth, while better approximating real-world attack surfaces.
- Multi-User Attacks and Hash Diversity: This study focused on a fixed-user attack with SHA-1 hashing. Future work should investigate multi-user database attacks, reuse patterns across accounts, and the effects of using modern hash functions (e.g., bcrypt, scrypt, Argon2), which are designed to be resistant to high-throughput guessing, including quantum-enhanced attacks.

The complete source code used in this project is available at: https://github.com/hargrh/QuantumPasswordCracking.git [5].

## References

[1] L. K. Grover, "A fast quantum mechanical algorithm for database search," in Proceedings of the 28th Annual ACM Symposium on the Theory of Computing (STOC '96), pp. 212–219, 1996.

[2] M. Dürmuth, M. Golla, P. Markert, A. May, and L. Schlieper, "Towards Quantum Large-Scale Password Guessing on Real-World Distributions." Available: https://eprint.iacr.org/2021/1299.pdf

[3] G. K. Zipf, *The Psycho-Biology of Language: An Introduction to Dynamic Philology*, Abingdon, Oxon: George Routledge And Sons, Ltd., 1936. Reprinted 2002, transferred to digital printing 2007 by Routledge, Taylor And Francis Group.

[4] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *IACR Cryptology ePrint Archive*, Report 2014/631, 2014. [Online]. Available: https://eprint.iacr.org/2014/631.pdf

[5] R. Cenzano, H. Hargreaves, J. Kim, T. Tylik, and J. Youngbar, "Quantum Password Cracking (Code Repository)," GitHub, 2024. [Online]. Available: https://github.com/hargrh/QuantumPasswordCracking.git